UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

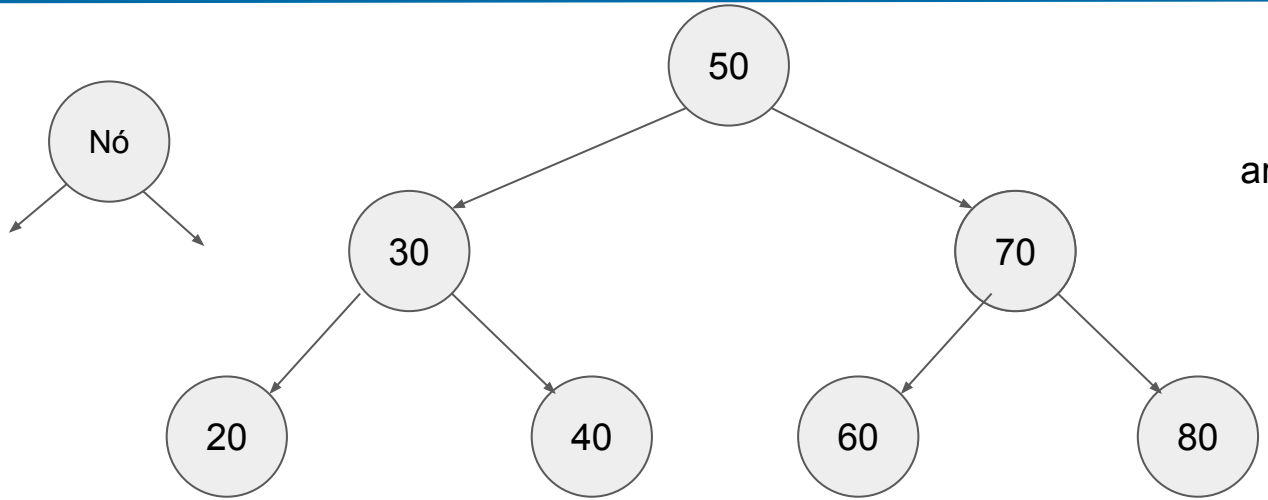
Disciplina: **Algoritmos e Programação (APE)**

Aula: 04 - Árvores (continuação)

IA PARA ENGENHEIROS
UFRGS

Professor: **Raphael Martins Brum**

2026



arvore.remover(80)

Pré ordem [prefix] multiplica(soma(20,40), subtracao(60 80))
Em ordem [infix] (20 + 40) * (60 - 80)
Pós ordem [postfix] 20 40 + 60 80 - * (notação RPN)

Exercício

1) Definir as classes para construirmos uma árvore binária de busca.
2) Definir métodos para inserir valores de uma lista numérica nesta árvore, respeitando a seguinte lógica:

   Valor do Filho 1 < Valor do Pai
   Valor do Filho 2 > Valor do Pai

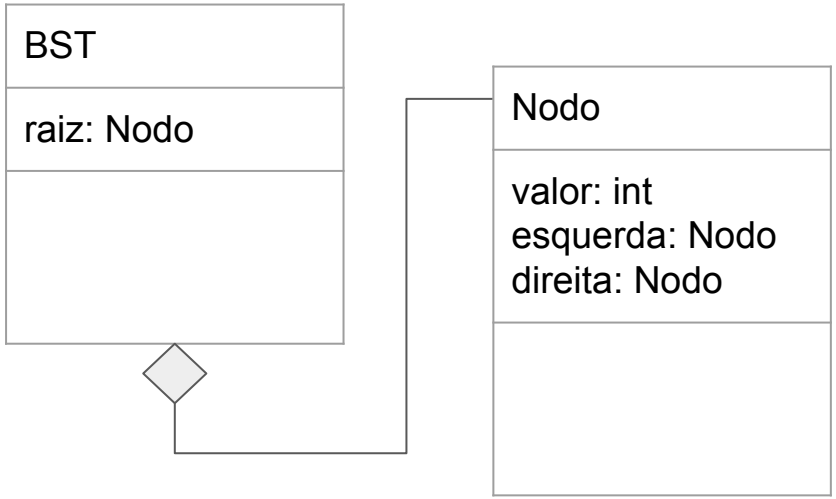
   Precisamos que um objeto “possua” outros objetos do mesmo tipo.
   (pai irá possuir os dois filhos: filho_esq, filho_dir)

3) Implementar uma função que percorre toda a árvore na ordem, exibindo o valor dos nós na sequência em que são visitados.

4) Implementar uma função que remove um valor da árvore.

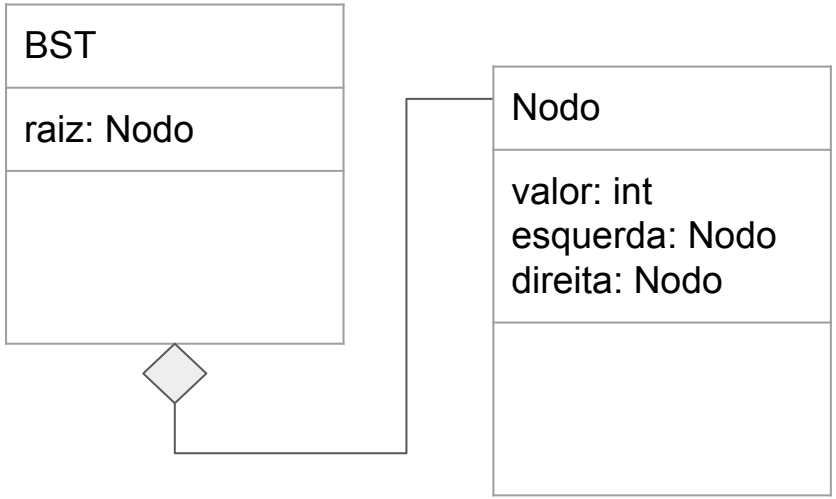
   (pensar o que fazer com os filhos!)

Definição das classes



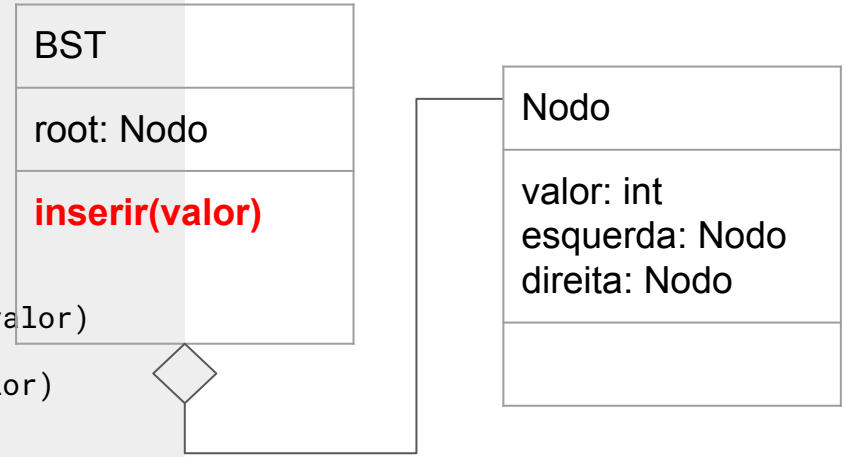
Definição das classes

```python
class BST:
  def __init__(self):
    self.raiz = None

class Nodo:
  def __init__(self, valor):
    self.valor = valor
    self.esquerda = None
    self.direita = None
```



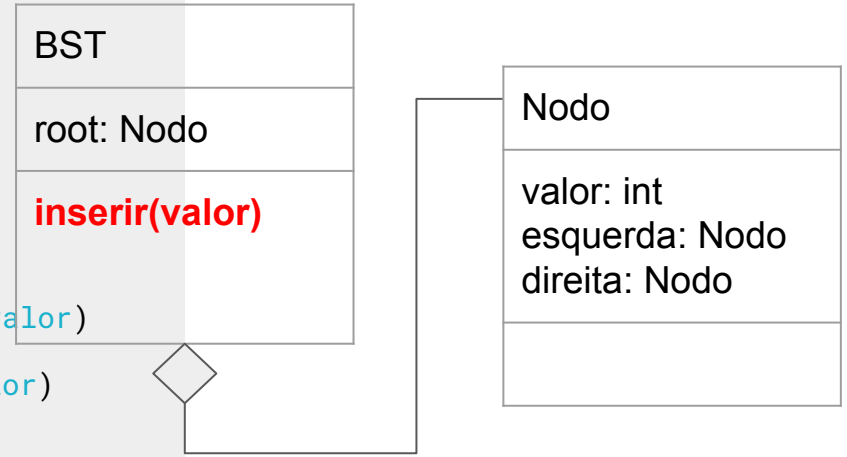
Inserir na árvore

```python
class BST:
(...)
  def inserir(self, valor):
    self.raiz = self._inserir(self.raiz, valor)

  def _inserir(self, nodo, valor):
    if nodo is None:
      return Nodo(valor)
    elif valor < nodo.valor:
      nodo.esquerda = self._inserir(nodo.esquerda, valor)
    elif valor > nodo.valor:
      nodo.direita = self._inserir(nodo.direita, valor)
    else:
      return nodo
```



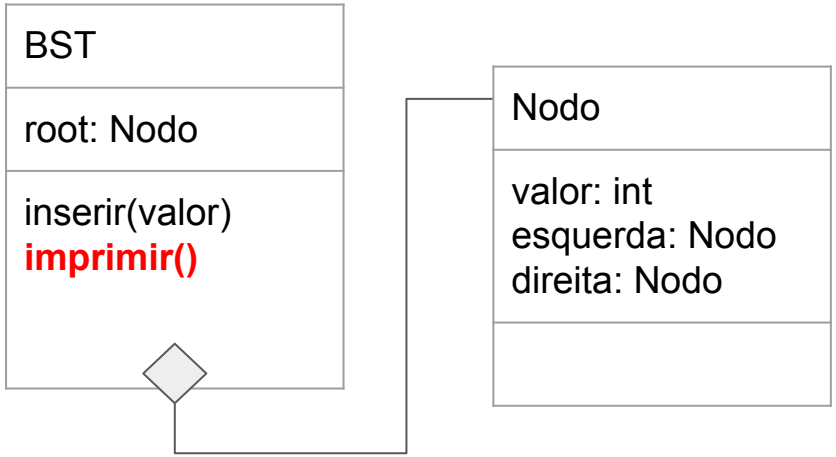
Inserir na árvore

```python
class BST:
(...)
  def inserir(self, valor):
    self.raiz = self._inserir(self.raiz, valor)

  def _inserir(self, nodo, valor):
    if nodo is None:
      return Nodo(valor)
    if valor < nodo.valor:
      nodo.esquerda = self._inserir(nodo.esquerda, valor)
    elif valor > nodo.valor:
      nodo.direita = self._inserir(nodo.direita, valor)
    return nodo
```



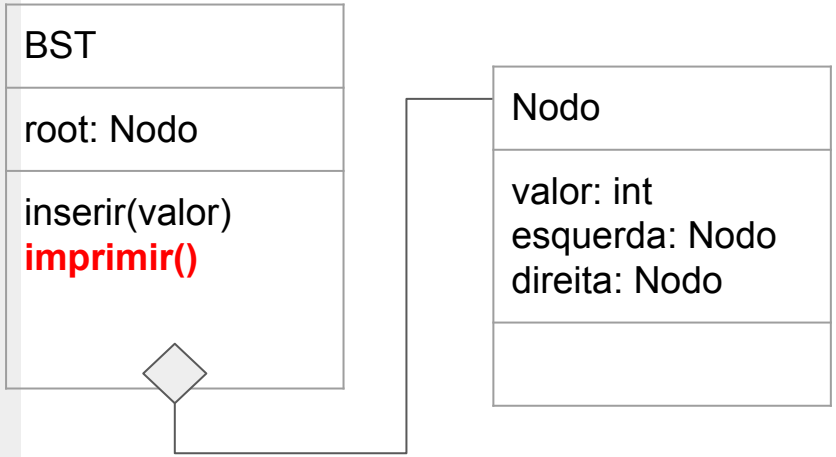
Exibir a árvore

```
class BST:
(...)
  def imprimir(self):
```



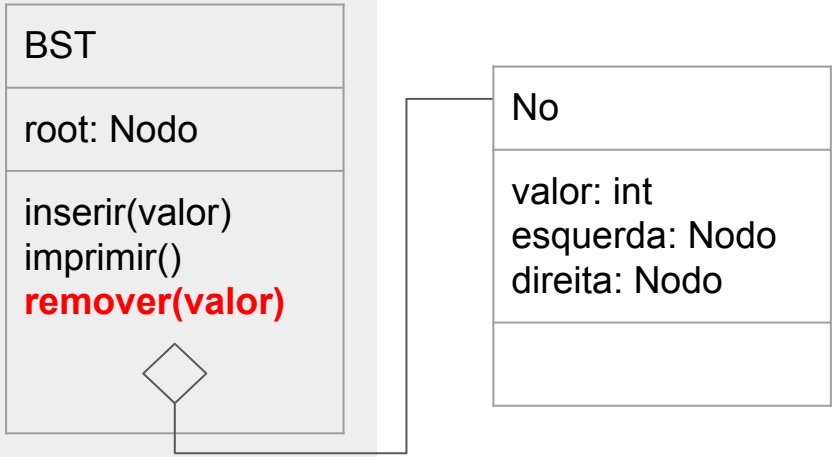
Exibir a árvore

```
class BST:
(...)
  def imprimir(self):
    if self.raiz is None:
        print("(árvore vazia)")
        return
    print(str(self.raiz.valor))
    self._imprimir(self.raiz.esquerda, "e:")
    self._imprimir(self.raiz.direita,  "d:")

  def _imprimir(self, nodo, indentacao):
      if nodo is None:
          return

      indentacao += "--"
      print(indentacao + str(nodo.valor))

      self._imprimir(nodo.esquerda, indentacao+"e:")
      self._imprimir(nodo.direita,  indentacao+"d:")
```



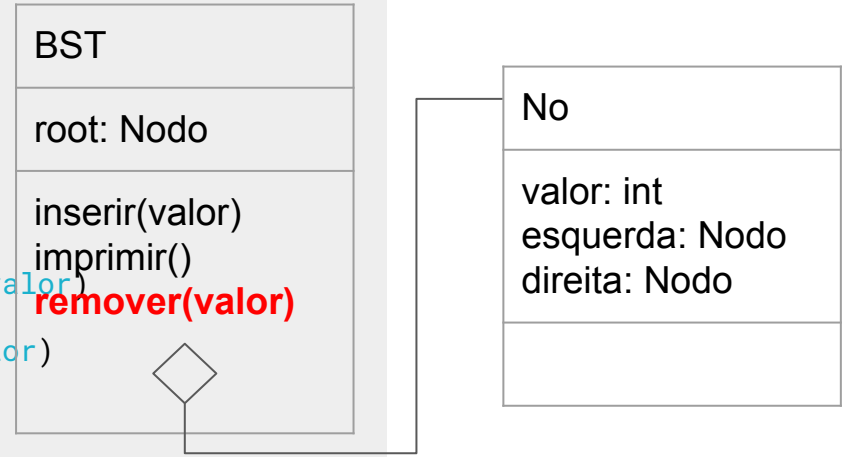
Remover da árvore

```
class BST:
  def remover(self, valor):
```



Remover da árvore

```python
class BST:
  def remover(self, valor):
    self.raiz = self._remover(self.raiz, valor)

  def _remover(self, nodo, valor):
    if nodo is None:
      return None
    if valor < nodo.valor:
      nodo.esquerda = self._remover(nodo.esquerda, valor)
    elif valor > nodo.valor:
      nodo.direita = self._remover(nodo.direita, valor)
    else:
      if nodo.esquerda is None:
        return nodo.direita
      if nodo.direita is None:
        return nodo.esquerda

      sucessor = self._minimo(nodo.direita)
      nodo.valor = sucessor.valor
      nodo.direita = self._remover(nodo.direita, sucessor.valor)
    return nodo

  def _minimo(self, nodo):
    while nodo.esquerda:
      nodo = nodo.esquerda
```



Travessias, caminhamento

- As operações nas árvores requerem que realizemos travessias (*searches, traversals*) através da estrutura de dados
- As travessias são classificadas em dois tipos:
  - DFS (*depth-first search*), ou busca em profundidade
  - BFS (breadth-*first search*), ou busca em largura

Depth-First Search (DFS)

- Pré-ordem (nó, esquerda, direita)
  - **Raiz aparece primeiro**
  - Árvore é visitada na ordem de inserção que a torna equilibrada
  - Aplicações: copiar a árvore, serializar a árvore.

- Em-ordem (esquerda, nó, direita)
  - **Visita os valores em ordem crescente na BST**

- Pós-ordem (esquerda, direita, nó)
  - Útil para remover elementos da árvore

Breadth-first Search (BFS)

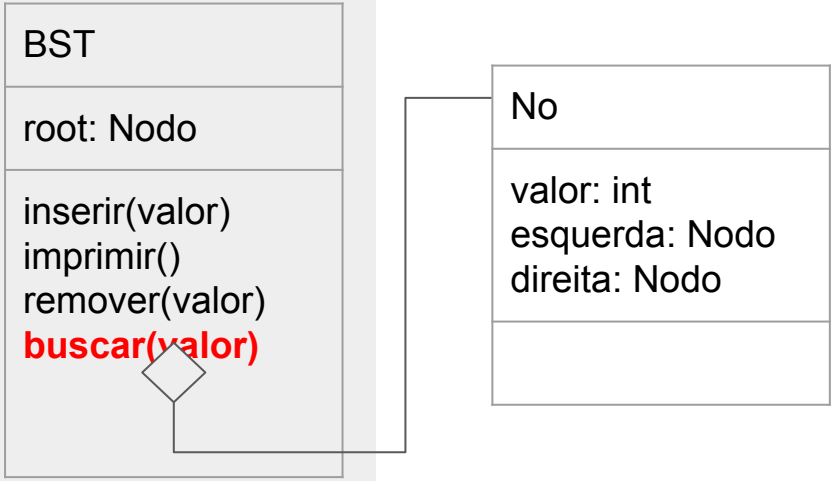
- Visita todos os nós de um dado nível antes de descer
- Mais facilmente implementável utilizando uma pilha
- Veremos adiante sua aplicação com grafos

Exercício

- Implementar as três travessias DFS, imprimindo o valor do nó a cada visita
- Implementar a travessia BFS, imprimindo o valor do nó a cada visita

Árvore binária de… busca!

```python
def buscar(self, valor):
  return self._buscar(self.raiz, valor)

def _buscar(self, nodo, valor):
  if nodo is None:
    return False
  if valor == nodo.valor:
    return True

  if valor < nodo.valor:
    return self._buscar(nodo.esquerda, valor)
  else:
    return self._buscar(nodo.direita, valor)
```

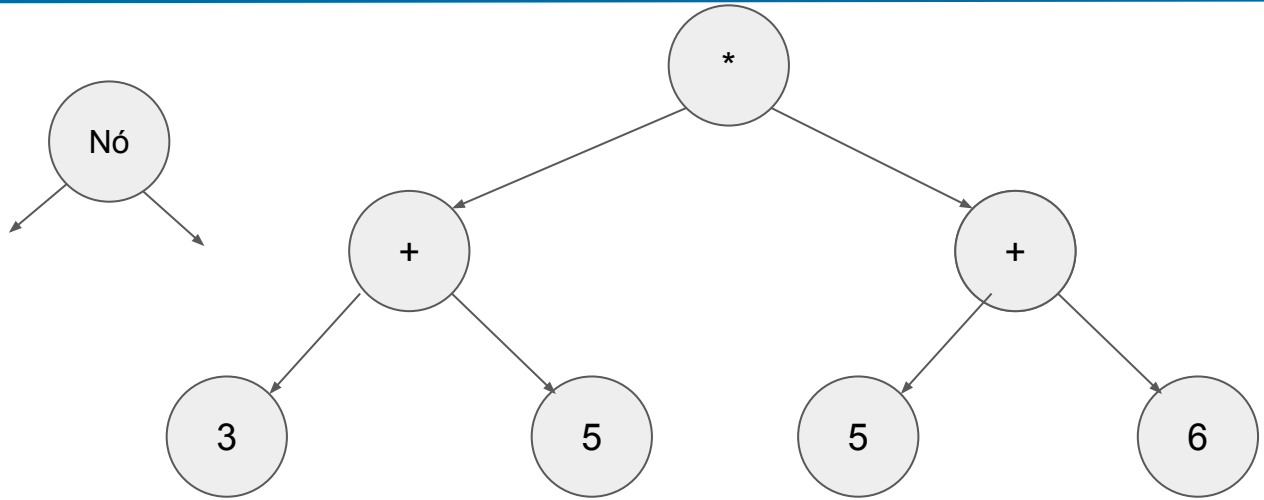


Aplicação da pós-ordem: calculadora RPN

- Notação Polonesa Reversa
  - Notação matemática em que os operadores são posicionados após os operandos (posfixa, em oposição à notação infixa comumente utilizada e à polonesa, prefixa)
  - Elimina a necessidade de parênteses
  - 2 + 2 → 2 2 +
  - (3 + 5) * (5 + 4) → 3 5 + 5 6 + *

Fila	Pilha
3	88
5	
+	
5	
6	
+	
*	

Exemplo de associação árvore / expressão



Caminhamento pós-fixado: 3 5 + 5 6 + *
Expressão correspondente: (3+5)*(5+6)
Resultado esperado: 8 * 11 = 88

Exercício

1) Fazer a leitura de uma “string” que representa um conjunto de operações em notação RPN.
2) Construir uma árvore (do tipo BST) que representa as operações a serem feitas.
3) Caminhar na BST para enfilerar os operandos e operadores adequadamente.
4) Utilizar uma pilha para implementar o algoritmo de cálculo baseado na notação RPN.